

도구를 쓰는 AI

— Tool Calling과 ReAct 루프

오늘 다루는 것

1. 도구 호출은 실제로 무엇을 하는가
2. 스키마를 어떻게 써야 모델이 도구를 부르는가
3. ReAct 루프 — 한 번에 끝내지 않는다
4. 에이전트는 언제 멈추는가
5. 실습: 회의록에서 할 일 뽑기

모델은 함수를 실행하지 않는다

- 도구를 "준다"는 말 때문에 오해가 생긴다
- 모델이 하는 일은 **JSON을 뱉는 것**뿐
- 실행은 **우리 코드**가 한다
- 결과를 다시 모델에게 **돌려줘야** 대화가 이어진다

비유하면 메뉴판이다. 모델은 주문을 하고, 요리는 우리가 한다.

```
# 모델의 응답
{
  "tool_calls": [{
    "id": "call_abc123",
    "function": {
      "name": "extract_tasks",
      "arguments": "{\"text\": \"...\"}"
    }
  }]
}
```

```
# 실행은 우리가
result = extract_tasks(**args)
```

스키마 — required가 없으면 모델은 망설인다

```
tools = [{
  "type": "function",
  "function": {
    "name": "extract_tasks",
    "description": "회의록에서 할 일과 담당자를 추출한다", # 구체적인수록 호출률↑
    "parameters": {
      "type": "object",
      "properties": {
        "tasks": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "task": {"type": "string"},
              "owner": {"type": "string"},
              "due": {"type": "string"}
            },
            "required": ["task", "owner"] # ← 빠뜨리기 쉬움
          }
        }
      },
      "required": ["tasks"] # ← 여기도
    }
  }
}]
```

Agent **실무 팁** 3 description이 모호하면 모델이 도구를 안 쓰고 줄글로 답한다. "무엇을 하는 도구인지"를 한 문장으로 분명히 쓸 것.

ReAct — 한 번에 끝내지 않는다



- 한 번 호출로 끝내면 모델이 **결과를 보고 판단을 고칠 기회**가 없다
- Observation은 별도 함수가 아니라 **role이 "tool"인 메시지를 넣는 것**
- 이걸 빠뜨리면 모델은 자기가 이미 호출했다는 사실을 모른다 → **같은 도구를 계속 다시 부른다**

루프 코드

```

messages = [{"role": "user", "content": 회의록}]

for turn in range(MAX_TURNS):          # 종료 조건 ①
    resp = client.chat.completions.create(
        model=MODEL, messages=messages, tools=tools
    )
    msg = resp.choices[0].message

    if not msg.tool_calls:              # 종료 조건 ② 도구를 안 부름 = 끝났다
        return msg.content

    messages.append(msg)                # Action 기록

    for call in msg.tool_calls:
        result = TOOLS[call.function.name](
            **json.loads(call.function.arguments)
        )
        messages.append({               # Observation - 이걸 빠뜨리면 무한루프
            "role": "tool",
            "tool_call_id": call.id,
            "content": json.dumps(result, ensure_ascii=False)
        })

```

에이전트는 언제 멈추는가

종료 조건	내용	언제 쓰나
① 자연 종료	모델이 tool_calls 없이 최종 답변을 냄	정상 경로
② 턴 상한	max_turns 도달 (보통 5~10)	안전장치. 반드시 넣는다
③ 반복 감지	같은 도구 + 같은 인자가 연속으로 나옴	무한루프 차단
④ 목표 검증	결과가 조건을 만족하는지 확인 후 종료	품질 보장이 필요할 때

가장 흔한 사고는 ②를 안 넣는 것이다. 개발 중에는 반드시 낮은 값(3~5)으로 두고 시작할 것.

회의록에서 할 일 뽑기

입력

지난주 팀 회의록 텍스트 (약 800자)

발언자 구분 없음, 구어체

출력

[[task, owner, due]] 형태의 JSON 배열

확인할 것

- 논의만 하고 끝난 건 제외되는가
- 담당자 없는 항목을 지어내지 않는가
- max_turns 안에 끝나는가

```
# 실습 시작 코드
def extract_tasks(tasks):
    return {"count": len(tasks),
            "tasks": tasks}

TOOLS = {"extract_tasks": extract_tasks}
MAX_TURNS = 5

# TODO: 위 루프를 완성하고
#       회의록.txt 를 넣어 실행
```